BÖLÜM 3 / MODULE 3

K8s İş Yükleri & Dayanıklılık

K8s Workloads & Self-Healing Resilience

Ölümsüz uygulamalar mimarisi: Deployments, ReplicaSets, kesintisiz güncellemeler, Spring Boot Actuator sağlık probları, StatefulSet ve kaynak kotaları.

Immortal applications architecture: Deployments, ReplicaSets, zero-downtime rolling updates, Spring Boot Actuator health probes, StatefulSets, and resource limits.

13

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

ReplicaSet ve Deployment: Ölümsüz Pod'lar

ReplicaSet and Deployment: Self-Healing Pods

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Tek başına bir Pod sokakta sahipsiz kalmış bir kedi yavrusu gibidir; arabaya çarparsa (çökerse) kimse arkasından ağlamaz! Deployment ise o kediye bakan 24 saat nöbetteki koruyucu melek ordusudur. 'Ben burada her zaman 3 adet sağlıklı Spring Boot kedisi istiyorum' dersiniz. Biri uyuyakalır veya ölürse, Deployment anında yenisini doğurur!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

A bare Pod is an unprotected orphan: if it crashes, nobody sheds a tear! A Deployment is its 24/7 guardian angel. You declare: 'I want exactly 3 healthy Spring Boot pods at all times.' If one pod gets killed or its server explodes, the Deployment controller instantly breathes life into an identical replacement!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Üretim ortamında neredeyse HİÇBİR ZAMAN çıplak Pod (bare pod) oluşturulmaz; her zaman Deployment kullanılır. Deployment hiyerarşisi şöyledir: Deployment -> ReplicaSet -> Pods. Deployment, yeni sürümler çıktığında yeni bir ReplicaSet açarak eski ReplicaSet'teki pod'ları kademeli olarak azaltır (Rolling Update). `spec.replicas` sayısı etcd'deki Desired State'tir. Pod çöktüğünde ReplicaSet controller bunu anında fark edip yeni bir Pod talebi oluşturur.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Bare pods are strictly anti-pattern in production; Deployments are mandatory. The architecture follows a strict hierarchy: Deployment -> ReplicaSet -> Pods. The Deployment controller manages declarative updates by spinning up a new ReplicaSet while scaling down the old one. If a container crashes, the Kubelet or ReplicaSet reconciles Pod counts back to the declared replica integer.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# spring-boot-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: coffee-deployment
  labels:
    app: coffee-app
spec:
  replicas: 3 # 3 adet yedekli pod çalıştır
  selector:
    matchLabels:
      app: coffee-app
  template:
    metadata:
      labels:
        app: coffee-app
    spec:
      containers:
        - name: coffee-container
          image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
          ports:
            - containerPort: 8080
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Deployment'ın `matchLabels` ile Pod şablonundaki (`template.metadata.labels`) etiketler birebir AYNI olmalıdır! Uyuşmazsa Deployment oluşturduğu pod'ları tanıyamaz ve sonsuz döngüde pod basmaya başlar.

🇬🇧 WATCH OUT

The `spec.selector.matchLabels` MUST match `spec.template.metadata.labels` identically. A mismatch causes the controller to lose track of its pods and trigger endless creation loops.

🎯 Özet Çıkarımlar

- Deployment, pod'ların çökmesi durumunda anında kendi kendini iyileştirir.
- Arka planda ReplicaSet nesneleri sürüm geçmişini ve kopya sayısını yönetir.
- Tek bir komutla (`kubectl scale deployment ... --replicas=5`) anında ölçeklenir.

🎯 Key Takeaways

- Deployments guarantee self-healing: dead pods are instantly replaced.
- ReplicaSets handle the low-level replica counting and historical revisions.
- Scale your fleet on demand with `kubectl scale deployment --replicas=N`.

14

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

Sıfır Kesintiyle Güncelleme (Rolling Updates & Rollbacks)

Zero-Downtime Deployments: Rolling Updates & Rollbacks

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

200 km hızla giden bir yolcu treninde yolcuların çayı dökülmeden vagonları tek tek yenisiyle değiştirdiğinizi hayal edin! Rolling Update budur: 1 yeni vagon ekle, yolcuları oraya al, 1 eski vagonu emekli et. Yeni vagon bozuk çıkarsa tek bir düğmeye (`undo`) basarsınız ve saniyesinde eski vagonlara geri dönülür!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imagine replacing train cars on a locomotive speeding at 200 km/h without spilling a single passenger's hot tea! That is a Rolling Update: add 1 new modern carriage, move passengers, retire 1 old carriage. And if the new carriage has broken AC, press one panic button (`rollout undo`), and K8s reverts back instantly!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Deployment varsayılan strateji olarak `RollingUpdate` kullanır. İki kritik parametre vardır: 1) `maxSurge` (Güncelleme esnasında istenen kopya sayısının üzerine kaç ekstra pod açılabileceği; örn: %25 veya 1), 2) `maxUnavailable` (Güncelleme sırasında kaç pod'un geçici olarak devre dışı kalabileceği; 0 yapılırsa HİÇBİR ZAMAN servis kesintisi yaşanmaz). Hata durumunda `kubectl rollout undo deployment/coffee-deployment` ile anında önceki stabil ReplicaSet'e geri dönülür.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

RollingUpdate configuration governs zero downtime via two levers: 1) `maxSurge` (maximum pods created beyond desired replica target), 2) `maxUnavailable` (maximum pods that can be taken down during the roll). Setting `maxUnavailable: 0` ensures 100% capacity during deployments. Instant recovery is guaranteed via `kubectl rollout undo`.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Sıfır Kesinti Stratejisi Tanımı:
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0

# Sürüm Güncelleme:
kubectl set image deployment/coffee-deployment coffee-container=ghcr.io/eli5-k8s/spring-boot-k8s-eli5:2.0.0

# Durumu Canlı Takip Et:
kubectl rollout status deployment/coffee-deployment

# Hata Anında Tek Tuşla Geri Al (Rollback):
kubectl rollout undo deployment/coffee-deployment
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Eğer yeni sürümdeki Spring Boot uygulamanız veritabanında geriye dönük uyumsuz (breaking change) bir tablo şeması değişikliği yaparsa, K8s seviyesinde rollback yapsanız bile veritabanınız çökmüş kalır! Veritabanı şemaları her zaman geriye dönük uyumlu (expand and contract) olmalıdır.

🇬🇧 WATCH OUT

Kubernetes rollbacks cannot un-break a destructive database migration! Database schema changes must always maintain backward compatibility across rolling versions.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- `maxUnavailable: 0` ile sıfır kesintili (zero-downtime) sürümler garanti edilir.
- Tüm dağıtım adımları `kubectl rollout status` ile canlı izlenebilir.
- Tek bir komutla (`rollout undo`) saniyeler içinde eski sürüme dönülebilir.

- `maxUnavailable: 0` ensures capacity never dips below the replica threshold.
- Monitor rollout health live using `kubectl rollout status`.
- Instant rollbacks undo broken releases in seconds without redeploying code.

15

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

Spring Boot Sağlık Kontrolleri: Liveness, Readiness ve Startup Probes

Spring Boot Health Probes: Liveness, Readiness, and Startup

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Doktor muayenesi zamanı! K8s üç farklı muayene yapar: 1) Startup Probe: 'Bebek doğdu mu, nefes aldı mı? (Spring Boot JVM ısınana kadar sabret!)' 2) Liveness Probe: 'Hasta yaşıyor mu? Kalbi durduysa (kilitlendiyse) elektroşok ver ve pod'u yeniden başlat!' 3) Readiness Probe: 'Müşteri kabul edebilir mi? Veritabanı bağlantısı koptuysa, masaya müşteri göndermeyi geçici olarak durdur!'

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Time for the medical checkup! Kubernetes acts as a vigilant doctor with 3 stethoscopes: 1) Startup Probe: 'Has the baby woken up yet? (Hold your horses while the heavy Java JVM warms up!)' 2) Liveness Probe: 'Is the heart deadlocked? If flatlining, kill and reboot it!' 3) Readiness Probe: 'Can the kitchen take orders? If the database connection drops, temporarily stop routing diners here!'

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Spring Boot Actuator 2.3+ ile `/actuator/health/liveness` ve `/actuator/health/readiness` uç noktaları yerleşik olarak gelir. 1) Startup Probe: Ağır uygulamaların başlaması uzun sürerse liveness'ın pod'u erken öldürmesini engeller. 2) Liveness Probe: Thread deadlock veya bellek tıkanıklığında Kubelet pod'u yeniden başlatır (`restartCount++`). 3) Readiness Probe: Başarısız olursa pod ÖLDÜRÜLMEZ; sadece K8s Service Endpoints listesinden çıkarılır, böylece kullanıcılara 502/503 hatası gitmez.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Spring Boot Actuator natively exposes `/actuator/health/liveness` and `/actuator/health/readiness`. 1) Startup Probe protects slow-booting JVM apps from premature liveness kills. 2) Liveness probe failures instruct Kubelet to terminate and restart the container. 3) Readiness probe failures do NOT kill the container; instead, kube-proxy strips the Pod IP from the Service endpoint slice, shielding users from HTTP 500 errors.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# application.yml:
management:
  endpoint:
    health:
      probes:
        enabled: true

# deployment.yaml Probes Bloğu:
startupProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  failureThreshold: 30
  periodSeconds: 2 # 60 saniye boyunca açılışı bekle
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  periodSeconds: 5
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Liveness Probe içinde veritabanı veya dış servis kontrolü yapmayın! Veritabanınız çökerse, kümedeki 100 Spring Boot pod'unun tamamı aynı anda liveness hatası verip yeniden başlar ve tüm küme kilitlenir (Cascading Failure). Veritabanı kontrolü sadece Readiness probe'da olmalıdır!

🇬🇧 WATCH OUT

NEVER check external dependencies (database, Redis) in your Liveness Probe! If the database stumbles, all 100 app pods will simultaneously reboot in a catastrophic cascading failure.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Startup Probe JVM uygulamalarının açılışındaki erken ölümleri önler.
- Liveness kilitlenen uygulamayı yeniden başlatır (Restart).
- Readiness hazır olmayan pod'a trafiğin akmasını engeller (Zero 500 Errors).

- Startup probes give heavy JVM applications generous breathing room to initialize.
- Liveness restarts frozen or deadlocked application processes.
- Readiness isolates unready pods from live customer traffic safely.

16

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

StatefulSet: Veritabanları Neden Özel İlgi İster?

StatefulSets: Why Databases Demand Special Treatment

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Koyun sürüsü (Deployment) vs Safkan yarış atları (StatefulSet). Koyunların adı yoktur; hepsi birbirinin aynıdır, biri kaybolsa yerine rastgele bir koyun koyarsınız. Ama yarış atlarının adı (`postgres-0`, `postgres-1`), sabit bir ahırı (özel kalıcı diski) ve hiyerarşisi (1. at liderdir, diğerleri takipçidir) vardır! Veritabanları koyun değil, yarış atıdır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Cattle (Deployments) vs Thoroughbred Racehorses (StatefulSets). Cattle have no individual names; they are completely interchangeable. Racehorses have distinct names (`postgres-0`, `postgres-1`), assigned personal stables (dedicated permanent storage), and strict hierarchies (Leader vs Replica). Databases are pedigreed racehorses!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Deployment pod'ları rastgele isimler alır (`app-7d6f5-k9x2`) ve çöktüklerinde yeni bir rastgele isimle doğarlar. StatefulSet ise şunları garanti eder: 1) Öngörülebilir ve sabit isimler (`db-0`, `db-1`, `db-2`), 2) Sıralı başlatma ve kapatma (önce `db-0` hazır olur, sonra `db-1` başlar), 3) `volumeClaimTemplates` sayesinde her pod'a çöktüğünde bile peşinden gelen özel bir PersistentVolume tahsis edilir, 4) Headless Service ile her pod doğrudan kendi DNS adresiyle çağrılabilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

StatefulSets provide 4 crucial guarantees for stateful systems: 1) Deterministic, stable network identifiers (`stateful-0`, `stateful-1`), 2) Ordered, graceful deployment and scaling, 3) Dedicated persistent storage per replica via `volumeClaimTemplates`, 4) Headless Service integration providing stable per-pod DNS records for master-replica topology negotiation.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# statefulset-postgres.yaml (Özet)
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: "postgres-headless"
  replicas: 2
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
  volumeClaimTemplates:
    - metadata:
        name: db-storage
      spec:
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
            storage: 10Gi
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Bir StatefulSet'i sildiğinizde (`kubectl delete statefulset`), K8s veri kaybını önlemek için bağlı olan kalıcı diskleri (PVC) OTOMATİK OLARAK SİLMEZ! Diskleri elle silmeniz gerekir.

🇬🇧 WATCH OUT

Deleting a StatefulSet does NOT delete its associated PVC storage claims by design! Kubernetes protects database disks from accidental deletion.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- StatefulSet pod'ları sabit isim (`pod-0`, `pod-1`) ve sabit DNS alır.
- `volumeClaimTemplates` her pod kopyasına özel kalıcı disk bağlar.
- Veritabanı, Kafka ve Elasticsearch gibi durum tutan servisler için zorunludur.

- StatefulSet replicas possess persistent network hostnames and dedicated DNS.
- Each replica obtains its own dedicated PersistentVolume via templates.
- Essential for stateful clustered systems like PostgreSQL, Kafka, and Redis.

17

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

DaemonSet ve Job / CronJob: Nöbetçiler ve Gece Vardiyaları

DaemonSets, Jobs, and CronJobs: Watchmen and Night Shifts

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Üç farklı çalışan tipi: 1) DaemonSet: Her apartman bloğunun giriş kapısındaki sabit güvenlik görevlisidir (Her sunucuda istisnasız 1 adet çalışır; log ve virüs bekçisi). 2) Job: 'Git şu odayı boya ve işin bitince evine git' dediğiniz boyacıdır (Tek seferlik çalışır ve kapanır). 3) CronJob: Her gece saat 03:00'te gelip çöpleri toplayan gece temizlikçisidir.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Three specialized workers: 1) DaemonSet: The building security guard posted at every single entrance without exception (Runs exactly 1 pod per node for monitoring/logging). 2) Job: The painter hired to paint a wall and leave once finished (Run-to-completion batch). 3) CronJob: The night cleaner who arrives automatically every night at 3:00 AM to sweep the floors.

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

1) DaemonSet: Kümeye yeni bir Worker Node eklendiğinde K8s otomatik olarak o düğümde de 1 adet pod başlatır (FluentBit, Datadog ajanı, Calico node). 2) Job: Spring Boot veritabanı şema göçü (Flyway/Liquibase) gibi bir kez çalışıp başarıyla (exit code 0) tamamlanması gereken işleri yönetir. Başarısız olursa `backoffLimit` kadar tekrar dener. 3) CronJob: Linux crontab sözdizimiyle (`0 3 * * *`) periyodik Job'lar üretir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

1) DaemonSet guarantees that every eligible node runs a copy of a pod, auto-scaling as nodes join or leave (ideal for log collectors and CNI agents). 2) Job creates pods that run until a specified number of successful completions, managing retry backoffs for DB schema migrations. 3) CronJob schedules recurring batch jobs using standard 5-part crontab syntax.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# flyway-migration-job.yaml (Spring Boot DB Göçü)
apiVersion: batch/v1
kind: Job
metadata:
  name: coffee-db-migration
spec:
  backoffLimit: 3 # En fazla 3 kez tekrar dene
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - name: db-migrator
          image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
          command: ["java", "-jar", "app.jar", "--spring.flyway.migrate=true"]
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Job şablonunda `restartPolicy` varsayılan olarak 'Always' OLAMAZ! Bir Job ya 'OnFailure' ya da 'Never' olmalıdır, aksi takdirde biten iş sonsuza dek yeniden başlar.

🇬🇧 WATCH OUT

Job specs cannot use `restartPolicy: Always`! It must be `OnFailure` or `Never`, otherwise completed tasks loop infinitely.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- DaemonSet her sunucuda istisnasız 1 kopya çalıştırarak altyapıyı izler.
- Job'lar tek seferlik veritabanı göçleri ve veri işleme için idealdir.
- CronJob zamanlanmış periyodik rapor ve temizlik görevlerini otomatikleştirir.

- DaemonSets ensure uniform node coverage for infrastructure monitoring and logs.
- Jobs run tasks to completion with built-in retry policies.
- CronJobs reliably automate scheduled periodic batch workloads.

18

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

Kaynak Yönetimi: Requests ve Limits (Açgözlü Pod'ları Dizginleme)

Resource Management: Requests, Limits, and OOMKilled Taming

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Açık büfe restoran kuralı: 'Masanıza oturduğunuzda en az 1 dilim böreği garanti ediyoruz (Request). Ama tabağınıza 5 dilimden fazla koyarsanız (Limit), güvenlik gelir ve tabağınızı elinizden alıp sizi kapı dışarı atar (OOMKilled)!' Kaynak sınırları koymazsanız, obur bir Spring Boot pod'u tüm sunucunun RAM'ini yiyip diğer pod'ları açlıktan öldürür!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The Golden Buffet Rule: 'When seated, we guarantee you at least 1 steak (Request). But if you attempt to hoard more than 5 steaks (Limit), security grabs you by the collar and throws you into the street (OOMKilled)!' Without limits, a gluttonous Java pod will devour the entire server's RAM, starving innocent neighboring pods to death!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

K8s iki kaynak türünü yönetir: CPU (Sıkıştırılabilir kaynak: Limit aşılırsa süreç öldürülmez, CPU Throttling ile yavaşlatılır) ve Memory (Sıkıştırılamaz kaynak: Pod memory limitini 1 bayt bile aşarsa Linux OOM Killer devreye girer ve pod `Exit Code 137: OOMKilled` ile anında öldürülür). `requests` Scheduler'ın pod'u yerleştireceği düğümü seçerken baktığı garantili rezervasyondur. `limits` ise pod'un asla aşamayacağı tavandır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes differentiates compressible and incompressible resources: CPU is throttled via CFS bandwidth control when exceeding limits; Memory cannot be throttled—exceeding limits triggers an immediate Linux kernel SIGKILL (`Exit Code 137: OOMKilled`). `requests` dictate scheduling placement; `limits` set strict cgroup enforcement ceilings.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Pod Kaynak Tanımı:
resources:
  requests:
    cpu: "200m"     # 0.2 CPU Çekirdeği garanti et
    memory: "512Mi" # 512MB RAM garanti et
  limits:
    cpu: "1000m"    # En fazla 1 tam çekirdek kullanabilir (Aşarsa yavaşlar)
    memory: "1Gi"   # 1GB'ı aşarsa K8s ACIMASIZCA öldürür (OOMKilled)!

# JVM Bellek Ayarı (Limitin %75'ini Heap yap, gerisi Metaspace/Stack kalsın):
env:
  - name: JAVA_TOOL_OPTIONS
    value: "-XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0"
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

JVM varsayılan olarak sunucunun toplam fiziksel belleğini görebilir! Eğer `-XX:MaxRAMPercentage` veya `-Xmx` vermezseniz, Java 32GB'lık sunucunun 8GB'ını almaya çalışır, K8s limiti olan 1GB'a çarpıp anında OOMKilled olur!

🇬🇧 WATCH OUT

Without container-aware flags (`-XX:MaxRAMPercentage=75.0`), the JVM defaults to host RAM, swiftly violating the cgroup limit and triggering instant OOMKilled panics.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Requests zamanlama (scheduling) garantisidir; Limits ise azami tavandır.
- CPU aşımı yavaşlamaya (throttling), Bellek aşımı ani ölüme (OOMKilled) yol açar.
- Java uygulamalarında JVM bellek limitleri ile K8s limitleri tam uyumlu olmalıdır.

- Requests guarantee scheduling capacity; Limits enforce hard runtime caps.
- CPU spikes trigger throttling; Memory spikes trigger immediate OOMKilled termination.
- Always calibrate JVM MaxRAMPercentage to prevent unexpected container eviction.